PADRÃO DE ENGENHARIA UNIVERSAL

Guia de Uso — versão 3.1

Base normativa para construir qualquer sistema, app, SaaS ou site — com IA ou sem — com qualidade profissional, segurança em camadas e memória de projeto à prova de perda de contexto.

Repositório oficial (somente leitura):

https://github.com/Viny-Atriaon/padrao-engenharia-universal-publico

Licença: CC BY-ND 4.0 — uso livre com atribuição; sem redistribuição de versões modificadas.

1. O que é este padrão

É um conjunto de 4 artefatos que funcionam juntos como um sistema operacional de engenharia. Ele foi desenhado para o cenário real de hoje: quem escreve a maior parte do código é um agente de IA (Claude Code, Codex, Antigravity, Cursor, Gemini...), mas o resultado precisa ser profissional o suficiente para qualquer desenvolvedor humano auditar, entender e manter.

Arquivo	Papel	Quem lê
CONSTITUICAO-ENGENHARIA-UNIVERSAL.md	A NORMA: ~145 regras com ID estável, níveis de rigor e parâmetros de contexto	A IA (colada na sessão)
HANDBOOK-ENGENHARIA-UNIVERSAL.md	O PORQUÊ: 23 capítulos explicando cada regra, quando não aplicar e erros comuns	Você (consulta humana)
KIT-DE-PARTIDA-UNIVERSAL.md	O EXECUTÁVEL: templates prontos + 3 prompts coláveis (partida, retomada, instanciação)	Você e a IA
audit.sh	O FISCAL: script que verifica automaticamente as regras [M] em qualquer projeto	O terminal

Os três princípios que resumem tudo: **(1)** a memória do projeto mora no repositório, nunca no chat; **(2)** a ordem de construção é dados → segurança → domínio → API → front, com segurança transversal desde o primeiro commit; **(3)** "funciona" sem evidência colada não conta como verificado.

2. Como obter os arquivos

Opção A — clonar com git (recomendado):

```
git clone https://github.com/Viny-Atriaon/padrao-engenharia-universal-publico.git
cd padrao-engenharia-universal-publico
```

Opção B — baixar ZIP: abra o repositório no navegador, botão verde **Code** → **Download ZIP**, e extraia numa pasta sua.

Versões: cada release tem uma tag (ex.: v3.1). Para garantir que você está usando uma versão estável e citável, confira a tag:

```
git tag          # lista as versões
git checkout v3.1  # trava na versão 3.1
```

O repositório é **somente leitura** para o público: você pode clonar, baixar e usar à vontade, mas não é possível alterar nem apagar nada nele — só o mantenedor publica mudanças.

3. Começando um projeto novo (o fluxo completo)

3.1. Defina o porte do projeto

Porte	Exemplos	Kit exigido
site-estatico	landing page, portfólio, institucional	mínimo (10 min)
app-simples	app com backend leve, formulários, CRUD pequeno	padrão (30 min)
sistema	sistema completo com banco, auth e deploy próprio	completo (~2 h)
SaaS	produto com assinatura, planos e multi-cliente	completo + camada SaaS

3.2. Cole o Prompt de Partida na sessão do seu agente de IA

Abra o KIT-DE-PARTIDA-UNIVERSAL.md, seção 7, copie o prompt e preencha o nome e o porte. Ele instrui o agente a, ANTES de qualquer código:

- executar o checklist pré-construção do seu porte (Kit, seção 1);
- criar os arquivos de memória (AGENTS.md, ESTADO.md, docs/FASES.md) a partir dos templates;
- propor o plano em fases verticais e aguardar o seu OK;
- só então começar a Fase 1, fechando com checkpoint (commit + ESTADO.md).

3.3. Gere a Constituição instanciada (recomendado)

Em vez de colar a Constituição inteira (~145 regras) em toda sessão, use o prompt da seção 9 do Kit: ele gera um docs/CONSTITUICAO-INSTANCIADA.md só com as regras do SEU contexto (porte, stack, time). Menos contexto gasto, mais obediência do agente. Regenere se os parâmetros mudarem.

3.4. Construa por fases, na ordem certa

O plano (docs/FASES.md) já nasce na ordem correta: F1 kit → F2 modelo de dados → F3 fundação de segurança (auth + validação + posse) → F4+ fatias verticais do produto. Cada fase cabe numa sessão e termina com algo demonstrável. Front antes do contrato de API é proibido; "segurança depois" é proibido.

4. Retomando um projeto (a prova anti-amnésia)

Sessão nova, outro dia, outra IA, contexto zerado? Cole o prompt de retomada (Kit, seção 7). O agente lê AGENTS.md e ESTADO.md, responde em 3 linhas onde parou, o que vem agora e se há bloqueio — e continua do ponto exato. Se ele precisar perguntar "onde paramos?", o ESTADO.md do projeto reprovou e deve ser corrigido.

5. Auditando qualquer projeto com o audit.sh

O script confere automaticamente as regras verificáveis por máquina e imprime um relatório com ✓ conforme, ✗ violação e alertas. Funciona em qualquer projeto (Windows via Git Bash, Linux e macOS):

```
bash audit.sh /caminho/do/seu/projeto
bash audit.sh /caminho/do/seu/projeto --deps   # inclui auditoria de dependências
```

Exemplo de saída:

```
============================================================
 AUDITORIA - Constituicao Universal v3.1 - meu-projeto
============================================================
[OK]  GIT-INIT-01            repositorio git presente
[OK]  SECRETS-GITIGNORE-01   .env nao rastreado
[X]   CTX-ESTADO-01          sem ESTADO.md - a memoria viva nao existe
[OK]  PIPE-SUPPLY-01a        lockfile presente (pnpm-lock.yaml)
[!]   DEP-QUARENTENA-01      ignore-scripts nao configurado
============================================================
 RESULTADO: 13 OK  .  1 X  .  7 !
```

Violações viram pendências registradas no AGENTS.md do projeto. Rode a auditoria a cada marco e antes de qualquer entrega.

6. As regras de ouro (resumo de bolso)

Regra	O que significa na prática
Kit antes de código	Nenhuma linha de código antes dos arquivos de memória existirem (CTX-KIT-01).
Checkpoint = savegame	Fase fechada = commit + ESTADO.md atualizado. Contexto ficando longo? Checkpoint imediato.
Decisão vira arquivo	Decisão tomada no chat é gravada em ADR/ESTADO antes da sessão acabar — ou não existe.
Ordem de construção	Dados > segurança > domínio > API > front. Segurança é transversal, nunca 'depois'.
Escala em duas moedas	Stateless, paginação e índices desde o dia 1 (barato); load balancer e réplicas só com dor medida.
Doc nasce com a feature	Endpoint criado = endpoint documentado no mesmo checkpoint. 'Documentar no final' é proibido.
Evidência ou não aconteceu	'Testei e funciona' exige a saída do teste colada.
Quarentena de dependências	Release com menos de ~14 dias não entra (exceto patch de segurança). Só registro oficial.
Código para humanos	Teste dos 10 minutos: um dev que nunca viu o projeto entende qualquer módulo só com código + REA

7. O que você pode e não pode fazer (licença)

Licença: **Creative Commons BY-ND 4.0** (Atribuição — Sem Derivações).

PODE:

- usar em qualquer projeto — pessoal, comercial, de clientes — sem pedir permissão;
- clonar, baixar e compartilhar os arquivos originais, com crédito ao repositório;
- instanciar a Constituição para os seus projetos (preencher parâmetros é uso, não modificação);
- adaptar internamente para o seu time/empresa (adaptações de uso privado são permitidas).

NÃO PODE:

- publicar ou redistribuir versões modificadas do padrão como se fossem o original;
- remover a atribuição;
- alterar ou apagar o repositório oficial (tecnicamente impossível — somente leitura).

Sugestões e correções são bem-vindas: abra uma Issue no repositório. Mudanças aceitas entram numa próxima versão oficial, com tag nova — o padrão evolui por prática real, nunca por acúmulo de teoria.